

TRƯỜNG ĐẠI HỌC THỦ DẦU MỘT
VIỆN CÔNG NGHỆ SỐ

BÁO CÁO ĐỒ ÁN

HỌC PHẦN: []

GIÁM SÁT VÀ ĐẬP KIỆN HÀNG

Sinh viên thực hiện:

STT	Họ và tên	MSSV	Lớp
1	[]	[]	[]
2	[]	[]	[]
3	[]	[]	[]

Lớp học phần: []

Nhóm: 9

Giảng viên hướng dẫn: []

TP. HỒ CHÍ MINH, NĂM 2026

BẢNG PHÂN CÔNG THÀNH VIÊN

Nhóm thực hiện: 9. Học phần: []. Lớp học phần: [].

Tên đề tài: Giám sát và đập kiện hàng.

STT	Họ tên / MSSV	Nhiệm vụ thực hiện	Mức đóng góp
1	[] / []	[]	[]
2	[] / []	[]	[]
3	[] / []	[]	[]

Các trường [] được giữ để nhóm bổ sung thông tin chính xác trước khi nộp. Số dòng thành viên có thể điều chỉnh theo danh sách thực tế; bảng chưa xác nhận phần việc hoặc tỷ lệ đóng góp của bất kỳ cá nhân nào.

Phạm vi báo cáo

Báo cáo trình bày mô hình IoT/AIoT với một ESP32-S3, cảm biến chuyển động, LCD1602 và buzzer, cùng nền tảng MQTT, backend, database và dashboard. Cấu trúc gồm ba chương theo mẫu 3 của hướng dẫn học phần.

Kết quả thực nghiệm dùng nhật ký kiểm thử và dữ liệu thật đã lưu của dự án, đối chiếu ngày 06/10/2026. Bộ dữ liệu được chọn gồm 74 đoạn hợp lệ trên breadboard; hệ thống chưa có mô hình AI đã huấn luyện. Các đề xuất về thử kiện hàng hoàn chỉnh và đánh giá mô hình được trình bày như hướng phát triển.

Thông tin hành chính cần hoàn thiện

Tên học phần: []; mã lớp học phần: []; giảng viên hướng dẫn: []; danh sách thành viên, MSSV, lớp và phân công: [].

MỤC LỤC

MỞ ĐẦU.....	1
CHƯƠNG 1. TỔNG QUAN HỆ THỐNG VÀ CÔNG NGHỆ LIÊN QUAN.....	2
1.1. Bài toán và mục tiêu của hệ thống.....	2
1.2. Thiết bị nhúng và cảm biến chuyển động.....	3
1.3. Truyền dữ liệu bằng MQTT.....	4
1.4. Lưu trữ, dịch vụ web và bảo vệ truy cập.....	5
1.5. Cơ sở phân loại theo ngưỡng và hướng bổ sung AI.....	6
CHƯƠNG 2. THIẾT KẾ PHẦN CỨNG, PHẦN MỀM VÀ MÔ HÌNH HOẠT ĐỘNG.....	7
2.1. Kiến trúc tổng thể và yêu cầu chức năng.....	7
2.2. Thiết kế phần cứng và ánh xạ chân.....	8
2.2.1. Liên kết các khối và bố trí cảm biến.....	9
2.3. Thiết kế firmware và luồng lấy mẫu.....	10
2.3.1. Phát hiện trạng thái và gom đợt va đập.....	11
2.4. Truyền sự kiện và xác nhận lưu dữ liệu.....	12
2.5. Thiết kế dữ liệu và API.....	13
2.6. Thiết kế giao diện giám sát.....	14
2.7. Quy trình dữ liệu và mô hình AI dự kiến.....	15
CHƯƠNG 3. TRIỂN KHAI, THỬ NGHIỆM VÀ ĐÁNH GIÁ HỆ THỐNG.....	16
3.1. Môi trường triển khai và phương pháp thử.....	16
3.2. Kết quả kiểm thử phần mềm và vận hành.....	17
3.3. Kết quả thu dữ liệu cảm biến.....	18
3.4. Đánh giá kết quả và giới hạn.....	19
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	20
TÀI LIỆU THAM KHẢO.....	21
PHỤ LỤC. HƯỚNG DẪN DEMO VÀ ĐỐI CHIẾU.....	23
A. Trình tự trình diễn.....	23
B. Tập phục vụ kiểm tra báo cáo.....	23
C. Phạm vi công cụ hỗ trợ.....	23

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Diễn giải
IoT	Internet of Things – Internet vạn vật
AI / AIoT	Trí tuệ nhân tạo / IoT có ứng dụng AI
IMU	Inertial Measurement Unit – cảm biến quán tính
MQTT	Giao thức truyền bản tin theo publish/subscribe
FIFO	First In, First Out – bộ đệm vào trước, ra trước
I ² C	Inter-Integrated Circuit – bus giao tiếp hai dây
GPIO	General Purpose Input/Output – chân vào/ra
NVS	Non-Volatile Storage – lưu trữ không mất khi tắt nguồn
RMS	Root Mean Square – căn trung bình bình phương
API	Application Programming Interface – giao diện lập trình
CSRF	Cross-Site Request Forgery – giả mạo yêu cầu liên trang
BSSID	Địa chỉ nhận diện một điểm truy cập Wi-Fi
RSSI	Received Signal Strength Indicator – mức tín hiệu thu
CSV	Comma-Separated Values – dữ liệu văn bản dạng bảng
NTP	Network Time Protocol – đồng bộ thời gian qua mạng

Đơn vị sử dụng: g là đơn vị gia tốc quy chiếu theo trọng trường; °/s là tốc độ góc; ms là mili giây; Hz là số mẫu hoặc chu kỳ trong một giây. Giá trị RSSI thường biểu diễn bằng dBm. Khối lượng mô hình được mô tả bằng gam (g), khác với g trong chỉ số gia tốc.

DANH MỤC BẢNG VÀ HÌNH

Danh mục bảng

Bảng 1.1. Các mục tiêu và đầu ra của đề tài.....	2
Bảng 1.2. Cấu hình đo được sử dụng trong firmware.....	3
Bảng 1.3. Các luồng bản tin chính của dự án.....	4
Bảng 2.1. Các yêu cầu thiết kế chính.....	7
Bảng 2.2. Ảnh xạ tín hiệu theo firmware hiện tại.....	8
Bảng 2.3. Ngưỡng và điều kiện trong bộ phát hiện.....	11
Bảng 2.4. Nhóm đặc trưng đầu vào AI.....	15
Bảng 3.1. Kết quả kiểm thử tự động của backend.....	17
Bảng 3.2. Các lượt thu thực tế được lựa chọn.....	18
Bảng 3.3. Đối chiếu nhãn quan sát với nhãn quy tắc.....	19

Danh mục hình

Hình 2.1. Kiến trúc tổng thể hệ thống.....	7
Hình 2.2. Liên kết tín hiệu phân cứng.....	9
Hình 2.3. Luồng xử lý mẫu và sự kiện.....	10
Hình 2.4. Trình tự xác nhận lưu sự kiện.....	12
Hình 2.5. Các nhóm dữ liệu của hệ thống.....	13
Hình 2.6. Dashboard với các mục thu gọn.....	14
Hình 3.1. Số đoạn hợp lệ theo nhãn quan sát.....	18

MỞ ĐẦU

Trong quá trình bốc xếp và vận chuyển, một kiện hàng có thể chịu rung động, thay đổi tư thế hoặc các cú va đập. Quan sát tình trạng bên ngoài tại thời điểm nhận hàng chưa cung cấp đầy đủ diễn biến đã xảy ra trước đó. Đề tài Giám sát va đập kiện hàng được thực hiện nhằm xây dựng một mô hình có khả năng ghi nhận các dấu hiệu cơ học, cảnh báo và lưu lịch sử để người vận hành xem lại.

Mục tiêu của đề tài là kết nối cảm biến chuyển động với ESP32-S3, xử lý trạng thái trên thiết bị, hiển thị và phát cảnh báo tại chỗ, đồng thời truyền dữ liệu đến máy chủ. Máy chủ lưu dữ liệu trong PostgreSQL và cung cấp dashboard có đăng nhập, biểu đồ, lịch sử sự kiện, tra cứu theo thời gian và xuất báo cáo. Định vị theo môt Wi-Fi được bổ sung để mô tả khu vực mà thiết bị đã nhận diện.

Phần AI hướng đến phân biệt ba nhóm Bình thường, Rung lắc và Va đập từ các đoạn đo sáu trục. Hệ thống đã có quy trình thu, kiểm tra chất lượng, gắn nhãn, xuất dữ liệu và công cụ huấn luyện Random Forest. Tại thời điểm lập báo cáo, mô hình chưa được huấn luyện và nạp; các cảnh báo đang vận hành theo quy tắc cảm biến.

Phạm vi thực nghiệm hiện tại là một ESP32-S3 và bộ mạch breadboard khoảng 100 g, gồm mạch điều khiển, LCD, cảm biến và buzzer. Dữ liệu này phục vụ kiểm tra mô hình kỹ thuật, chưa đại diện cho kiện hàng đóng gói thực tế. Báo cáo sử dụng mã nguồn, nhật ký kiểm thử và dữ liệu đã được người thực hiện xác nhận đến ngày 06/10/2026 [1]; không đưa ra độ chính xác AI hoặc kết luận hư hại của hàng khi chưa có bằng chứng.

Báo cáo được tổ chức theo cấu trúc đề tài IoT/AIoT: Chương 1 trình bày tổng quan và công nghệ; Chương 2 mô tả thiết kế phần cứng, phần mềm và mô hình hoạt động; Chương 3 trình bày triển khai, thử nghiệm và đánh giá. Phần cuối nêu kết quả đạt được, hạn chế và hướng phát triển.

CHƯƠNG 1. TỔNG QUAN HỆ THỐNG VÀ CÔNG NGHỆ LIÊN QUAN

1.1. Bài toán và mục tiêu của hệ thống

Bài toán đặt ra là ghi nhận các trạng thái cơ học của đối tượng được gắn cảm biến và chuyển các kết quả đo thành thông tin có thể tra cứu. Một hệ thống phù hợp cần theo dõi liên tục tại thiết bị, lưu thời điểm xảy ra sự kiện, thông báo cho người dùng và hỗ trợ xem lại sau khi trạng thái tức thời đã trở về bình thường. Vì vậy, trạng thái hiện tại và lịch sử cảnh báo được thiết kế thành hai nội dung riêng.

Đề tài lựa chọn mô hình gồm thiết bị cảm biến, kênh truyền MQTT, máy chủ xử lý, cơ sở dữ liệu và web dashboard. ESP32 xử lý quy tắc cơ bản ngay tại chỗ để LCD và còi không phải chờ trình duyệt. Máy chủ tiếp nhận dữ liệu, chống lưu trùng và cung cấp các chức năng tổng hợp. Người dùng truy cập dashboard để xem thông số, xác nhận đã xem cảnh báo hoặc xuất dữ liệu phục vụ phân tích.

Bảng 1.1. Các mục tiêu và đầu ra của đề tài

Mục tiêu	Đầu ra cần có
Giám sát chuyển động	Gia tốc tổng hợp, góc nghiêng, mức rung và trạng thái.
Ghi nhận va đập	Đỉnh gia tốc, mức thuật toán, thời gian và thời lượng quan sát.
Cảnh báo	LCD, còi và thông báo trên dashboard.
Quản lý lịch sử	Lưu database, lọc, thống kê và xuất CSV.
Chuẩn bị AI	Các đoạn đo có nhãn quan sát, đặc trưng và công cụ đánh giá.

Nguồn: tổng hợp từ mã nguồn và dữ liệu dự án [1].

Bảng 1.1 cho thấy đầu ra được xác định theo khả năng quan sát và lưu thông tin, thay vì theo kết luận hàng đã hỏng. Mức Nhẹ, Trung bình và Mạnh hiện là các khoảng gia tốc của thuật toán. Việc xác định hư hại cần bổ sung loại hàng, cách đóng gói và nhãn kiểm tra sản phẩm sau vận chuyển.

Mô hình hiện triển khai cho thiết bị esp32-001. Các thành phần phần mềm có sử dụng mã thiết bị trong API và bản tin, nhưng báo cáo chỉ đánh giá cấu hình một ESP32 đã được kiểm tra. Năng lực vận hành nhiều thiết bị hoặc ngoài mạng thử hiện chưa được xác nhận bằng thực nghiệm tải.

1.2. Thiết bị nhúng và cảm biến chuyển động

ESP32-S3 là vi điều khiển tích hợp kết nối Wi-Fi, bộ xử lý hai lõi và các ngoại vi như GPIO, I²C, UART và USB. Những khả năng này phù hợp với mô hình vừa đọc cảm biến, điều khiển cảnh báo, vừa trao đổi dữ liệu mạng [2]. Trong đề tài, Wi-Fi được dùng để kết nối MQTT; không sử dụng Bluetooth để truyền dữ liệu giám sát.

Cảm biến thực tế được firmware nhận diện qua WHO_AM_I = 0x70, tương thích MPU6500. MPU6500 kết hợp gia tốc kế và con quay hồi chuyển ba trục, có bộ đệm FIFO và các thang đo cấu hình được [3]. Driver của dự án hỗ trợ nhận diện MPU6050/MPU6500 nhưng cấu hình thực nghiệm trong báo cáo là cảm biến tương thích MPU6500.

Gia tốc ba trục phản ánh tín hiệu mà cảm biến quan sát theo các trục gắn với bộ mạch. Con quay hồi chuyển cung cấp tốc độ góc, có ích khi mô tả thao tác xoay hoặc rung lắc. Khi đặt yên, độ lớn gia tốc tổng hợp thường gần 1 g do thành phần trọng lực; vì vậy một giá trị gần 1 g không có nghĩa là thiết bị đang chịu cú va đập.

Bảng 1.2. Cấu hình đo được sử dụng trong firmware

Thông số	Giá trị cấu hình hoặc nhận diện
Thiết bị điều khiển	ESP32-S3; mã thiết bị esp32-001.
Cảm biến đang thử	MPU6500-compatible; WHO_AM_I = 0x70.
Thang gia tốc	± 16 g trên từng trục; 2048 LSB/g.
Thang tốc độ góc	± 500 °/s; 65,5 LSB/(°/s).
Nhịp danh định	100 Hz; đọc dữ liệu bằng FIFO.
Đoạn đo dành cho AI	200 mẫu, tương đương khoảng 2 giây.

Nguồn: tổng hợp từ mã nguồn và dữ liệu dự án [1].

Các giá trị trong Bảng 1.2 mô tả cấu hình đã được lập trình, không phải chứng nhận hiệu chuẩn của cảm biến. Trục tọa độ phụ thuộc cách gắn; khi chuyển từ breadboard sang hộp kiện, cần ghi lại hướng gắn và lấy lại tư thế chuẩn. Tổng hợp gia tốc cũng có thể lớn hơn 16 g mặc dù thang ± 16 g áp dụng riêng cho từng trục.

Việc đọc FIFO giảm phụ thuộc vào thời điểm tác vụ phần mềm kịp đọc từng mẫu. Tuy nhiên, FIFO vẫn có dung lượng hữu hạn và dữ liệu không kèm dấu thời gian phần cứng cho từng mẫu trong cấu hình đang dùng. Những yếu tố này được tính đến trong kiểm tra chất lượng và phạm vi kết luận của báo cáo.

1.3. Truyền dữ liệu bằng MQTT

MQTT tổ chức trao đổi theo mô hình xuất bản/đăng ký. Thiết bị xuất bản bản tin lên topic; broker tiếp nhận và chuyển đến các bên đăng ký. Theo đặc tả MQTT 3.1.1, QoS 1 hỗ trợ giao nhận ít nhất một lần, nên bản tin có thể được giao lại [4]. Do đó, hệ thống vẫn phải xử lý bản trùng và không thể dùng số lần nhận bản tin để thay cho số sự kiện thực.

Trong kiến trúc này, ESP32 không cần gửi trực tiếp đến trình duyệt. EMQX là broker trung gian; backend đăng ký các topic của thiết bị và xử lý dữ liệu đến. Dashboard lấy kết quả từ REST API, giúp phân hiển thị không phụ thuộc vào chi tiết kết nối MQTT. Cách tổ chức này cũng tạo một vị trí tập trung để kiểm tra định dạng và lưu dữ liệu.

Bảng 1.3. Các luồng bản tin chính của dự án

Nhóm topic	Nội dung và mục đích
telemetry	Chỉ số tức thời, trạng thái, RSSI và bộ đếm đồng bộ.
event / event-ack	Sự kiện có ID và xác nhận đã lưu từ backend.
location-scan	Các điểm truy cập Wi-Fi được thiết bị quan sát.
motion-window / motion-ack	Đoạn 200 mẫu và xác nhận lưu đoạn thu.
Các topic lệnh thu	Điều khiển bắt đầu/dừng theo mã lần thu.

Nguồn: tổng hợp từ mã nguồn và dữ liệu dự án [1].

Các topic dữ liệu được đặt dưới tiền tố smart-package/esp32-001/. Telemetry phục vụ quan sát mới nhất, trong khi event phục vụ lịch sử bền vững. Các đoạn chuyển động dùng cho xem trực tiếp và cho phiên thu có nhãn được quản lý riêng. Phân tách này tránh coi mọi dữ liệu trực tiếp là mẫu huấn luyện đã được kiểm chứng.

PUBACK chỉ xác nhận một bước giao nhận ở mức MQTT. Dự án bổ sung receipt SAVED do backend gửi sau khi giao dịch database hoàn tất. ESP32 chỉ đánh dấu sự kiện đã đồng bộ khi nhận receipt phù hợp với mã thiết bị và ID sự kiện. Nếu chưa có receipt, sự kiện tiếp tục được gửi lại theo chu kỳ của ứng dụng.

Tốc độ xử lý hàng đợi cũng ảnh hưởng đến tính kịp thời. Trong quá trình kiểm tra, khoảng chờ MQTT 1.000 ms làm bản tin tích tụ khi có telemetry và các đoạn đo. Cấu hình được điều chỉnh còn 50 ms; chi tiết trước và sau sửa được trình bày trong Chương 3. Đây là thay đổi của triển khai, không phải thay đổi ý nghĩa QoS trong giao thức.

1.4. Lưu trữ, dịch vụ web và bảo vệ truy cập

NVS của ESP-IDF cung cấp khả năng lưu giá trị trong flash để đọc lại sau khởi động [5]. Đề tài sử dụng NVS cho cấu hình và lịch sử sự kiện tại ESP32. Để hạn chế chặn luồng đọc cảm biến, bản ghi được chuyển qua hàng đợi RAM rồi mới ghi flash. Chỉ phần đã ghi thành công mới được xem là tồn tại bền vững trên thiết bị.

Backend được xây dựng bằng Spring Boot, xử lý MQTT và cung cấp REST API. Các thao tác trên dashboard được bảo vệ bằng phiên đăng nhập. Spring Security cung cấp cơ chế bảo vệ CSRF đối với các phương thức có thể thay đổi dữ liệu [6]; trong dự án, frontend lấy token và gửi kèm yêu cầu ghi, gắn nhãn hoặc nạp mô hình.

PostgreSQL lưu thông số, sự kiện và dữ liệu thu. Ràng buộc duy nhất trên tổ hợp cột có thể ngăn các hàng trùng theo một khóa nghiệp vụ [7]. Dự án áp dụng tổ hợp mã thiết bị và ID sự kiện để việc gửi lại không tạo thêm sự kiện lịch sử. Khóa dữ liệu của đoạn AI sử dụng mã lần thu và số thứ tự đoạn.

React tổ chức giao diện thành các thành phần; Nginx phục vụ bản build và chuyển các yêu cầu /api đến backend. Cloudflare Tunnel kết nối dịch vụ tại máy chủ đến Cloudflare bằng kết nối hướng ra ngoài, hỗ trợ công bố dịch vụ mà không cần mở trực tiếp cổng máy chủ ra Internet [8]. Tên miền đang dùng là monitor.huan2k5.id.vn; khả năng truy cập còn phụ thuộc máy chủ và đường truyền đang hoạt động.

Mô hình lưu trữ phân biệt ba trạng thái: dữ liệu còn trong RAM, dữ liệu đã ghi NVS và dữ liệu đã lưu trên database. Ba trạng thái không có cùng mức bền vững. Khi hàng đợi hoặc lịch sử thiết bị không còn chỗ, hệ thống ghi nhận số sự kiện bị từ chối để người vận hành thấy giới hạn, thay vì ghi đè các sự kiện chưa được xác nhận.

Bộ đếm rejectedEvents là số tích lũy về các lần không lưu được sự kiện; pendingEvents là số sự kiện đang chờ đồng bộ. Giao diện hiển thị hai đại lượng này theo ý nghĩa riêng. Một tổng tích lũy khác 0 không đủ để kết luận bộ nhớ hiện tại đang đầy, còn việc xác nhận đã xem thông báo chỉ thay đổi trạng thái hiển thị trong trình duyệt.

Các thông tin mật khẩu, tài khoản kết nối và token triển khai được lấy từ tệp cấu hình môi trường. Báo cáo không chèn giá trị bí mật vào ví dụ hoặc ảnh. Những lớp bảo vệ hiện có phục vụ mô hình thử nghiệm; đánh giá tải lớn và kiểm thử bảo mật chuyên sâu nằm ngoài kết quả đã thực hiện.

1.5. Cơ sở phân loại theo ngưỡng và hướng bổ sung AI

Phân loại theo ngưỡng có ưu điểm dễ kiểm tra, chạy trực tiếp trên ESP32 và phản hồi độc lập với máy chủ. Hạn chế là các thao tác khác nhau có thể tạo tín hiệu gần nhau: rung mạnh có thể vượt ngưỡng va đập, còn một cú gõ nhẹ có thể chưa đủ để kích hoạt sự kiện. Vì vậy, cần đối chiếu dữ liệu đo với thao tác quan sát, thay vì mặc nhiên lấy kết quả quy tắc làm đáp án.

Với gia tốc đã đổi sang đơn vị g, độ lớn tổng hợp tại mẫu i được tính từ ba trục. Một đoạn N mẫu có thể được mô tả bằng đỉnh, trung bình, độ lệch chuẩn và RMS động. Các đặc trưng này phản ánh cả biên độ lẫn mức biến thiên của tín hiệu, nhưng chỉ mô tả những mẫu mà hệ thống đã quan sát.

$$G_i = \sqrt{(a_{x,i})^2 + (a_{y,i})^2 + (a_{z,i})^2}$$

$$\text{RMS động} = \sqrt{\{[(G_1 - 1)^2 + \dots + (G_N - 1)^2]/N\}}$$

Công cụ của dự án lựa chọn Random Forest. Phương pháp kết hợp nhiều cây quyết định để phân loại từ các đặc trưng đầu vào [9]. Trong thiết kế này, mô hình học trên máy tính và xuất cấu trúc cây JSON để backend Java suy luận. Đây là hướng AI trên máy chủ; ESP32 vẫn chịu trách nhiệm đo và cảnh báo theo ngưỡng.

Các đoạn liên tiếp của cùng lượt thu có thể tương quan mạnh. GroupShuffleSplit hỗ trợ chia dữ liệu theo nhóm thay vì chia ngẫu nhiên từng hàng [10]. Dự án dùng nhóm buổi thử để tránh đặt các đoạn gần nhau của cùng phiên vào cả tập huấn luyện và kiểm tra. Tên nhóm chỉ có ý nghĩa khi phản ánh một lần chuẩn bị hoặc điều kiện thử thực sự.

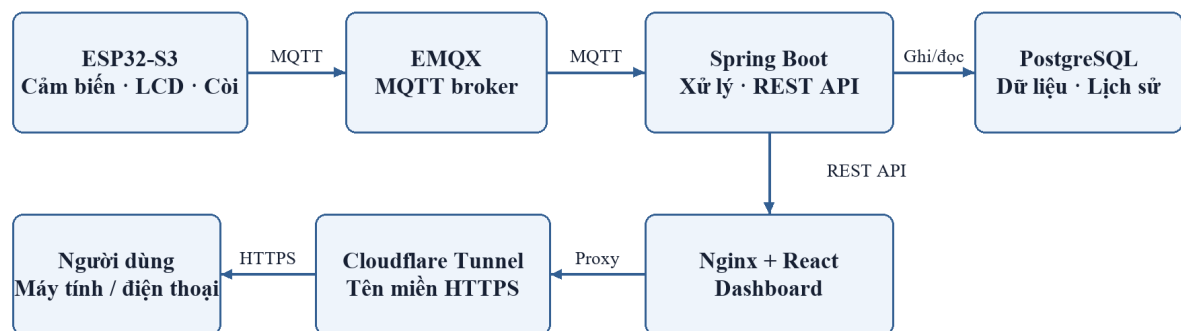
Ba nhãn AI dự kiến là Bình thường, Rung lắc và Va đập. Chúng không thay thế toàn bộ bảy trạng thái của firmware, cũng không mang ý nghĩa hư hại. Khi chưa có mô hình hoặc dữ liệu không hợp lệ, giao diện phải hiển thị tình trạng tương ứng thay vì tạo kết luận AI. Chỉ số đánh giá cần lấy từ tập kiểm tra chưa dùng để lựa chọn mô hình.

Việc có công cụ huấn luyện chưa đồng nghĩa đã có mô hình đạt yêu cầu. Báo cáo phân biệt rõ phần phương pháp đã lập trình, phần dữ liệu thật đã thu và phần đánh giá chưa thể thực hiện. Sự phân biệt này là cơ sở để lựa chọn các bước thử tiếp theo có ích cho đề tài.

CHƯƠNG 2. THIẾT KẾ PHẦN CỨNG, PHẦN MỀM VÀ MÔ HÌNH HOẠT ĐỘNG

2.1. Kiến trúc tổng thể và yêu cầu chức năng

Hình 2.1 trình bày kiến trúc tổng thể theo mã nguồn đang triển khai. Khối thiết bị đọc cảm biến và xử lý quy tắc; khối truyền thông đưa dữ liệu đến backend; khối lưu trữ duy trì lịch sử; khối giao diện hỗ trợ giám sát và thao tác. Luồng cảnh báo tại LCD/còi được thực hiện trên thiết bị, còn web phụ thuộc vào kết nối và nhịp cập nhật API.



Sự kiện được xác nhận sau khi giao dịch PostgreSQL hoàn tất.

Hình 2.1. Kiến trúc tổng thể hệ thống

Nguồn: sơ đồ/biểu đồ tổng hợp từ dự án [1].

Bảng 2.1. Các yêu cầu thiết kế chính

Yêu cầu	Cách đáp ứng
Cảnh báo tại chỗ	Xử lý trạng thái, LCD và còi trên ESP32.
Lưu sự kiện có kiểm soát	ID duy nhất, NVS, gửi lại và receipt sau lưu database.
Tra cứu dữ liệu	REST API, bộ lọc, phân trang, thống kê và CSV.
Thu mẫu cho AI	Đoạn 200 mẫu, nhãn quan sát và kiểm tra chất lượng.

Nguồn: tổng hợp từ mã nguồn và dữ liệu dự án [1].

Yêu cầu phi chức năng tập trung vào tính minh bạch của trạng thái dữ liệu và khả năng sử dụng. Dashboard phải báo mất kết nối khi thiếu dữ liệu mới, không tự tạo vị trí khi chưa khớp mốc, không hiển thị kết quả AI khi chưa có mô hình. Khi phát sinh lỗi lưu hoặc thiếu đoạn, dữ liệu đã nhận phải được giữ lại để xem xét.

Tổ chức bằng Docker Compose giúp các dịch vụ có cấu hình khởi động thống nhất. Việc tách nhiệm vụ cũng giúp kiểm tra lỗi theo từng lớp: cảm biến, Wi-Fi/MQTT, backend/database hoặc giao diện, thay vì xem mọi trường hợp thiếu hiển thị là lỗi của ESP32.

2.2. Thiết kế phần cứng và ánh xạ chân

Mô hình gồm ESP32-S3, cảm biến chuyển động, LCD1602, buzzer chủ động và breadboard. Phần cứng thực nghiệm được người thực hiện mô tả có khối lượng khoảng 100 g; cảm biến cố định trên breadboard. Cấu hình chân trong Bảng 2.2 được lấy từ main.c, lcd1602.c và buzzer.c, không suy đoán từ loại module thông dụng.

Bảng 2.2. Ánh xạ tín hiệu theo firmware hiện tại

Khối / tín hiệu	Chân ESP32-S3	Vai trò
Cảm biến – SDA	GPIO 8	Dữ liệu I ² C.
Cảm biến – SCL	GPIO 9	Clock I ² C.
LCD – RS	GPIO 4	Chọn lệnh hoặc dữ liệu.
LCD – E	GPIO 5	Xung cho phép ghi.
LCD – D4	GPIO 6	Bit dữ liệu LCD.
LCD – D5	GPIO 7	Bit dữ liệu LCD.
LCD – D6	GPIO 15	Bit dữ liệu LCD.
LCD – D7	GPIO 16	Bit dữ liệu LCD.
Buzzer	GPIO 17	Điều khiển còi chủ động.

Nguồn: tổng hợp từ mã nguồn và dữ liệu dự án [1].

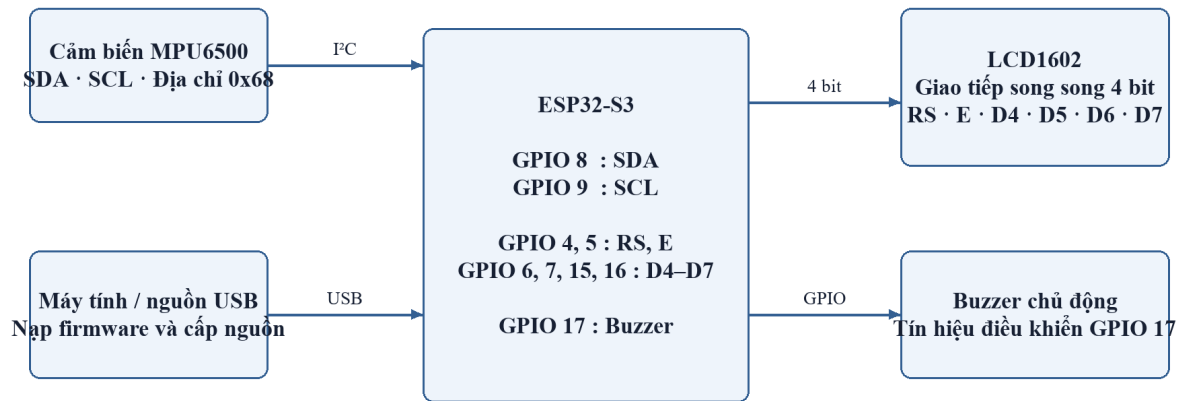
Cảm biến dùng địa chỉ I²C 0x68 và tốc độ bus cấu hình 400 kHz. LCD1602 được điều khiển bằng sáu đường GPIO ở chế độ dữ liệu 4 bit; driver không sử dụng bộ chuyển đổi I²C cho LCD. Khi thay loại module hiển thị, cần thay đổi driver và sơ đồ nối tương ứng.

Buzzer được điều khiển bằng mức GPIO và các mẫu phát tiếng trong tác vụ riêng. Khi nhận cảnh báo, tác vụ xử lý mẫu không phải chờ toàn bộ chuỗi còi kết thúc. Cách tổ chức này giúp giảm tác động của phần hiển thị và báo âm lên khả năng đọc dữ liệu cảm biến.

Bảng ánh xạ chỉ mô tả tín hiệu được phần mềm sử dụng. Điện áp cấp cho từng module, yêu cầu chuyển mức logic và các chi tiết đấu nối nguồn cần được đối chiếu với bộ mạch thực tế trước khi đưa vào thiết kế PCB. Báo cáo không thay một sơ đồ tín hiệu bằng khẳng định rằng mọi đặc tính điện đã được đo.

2.2.1. Liên kết các khối và bố trí cảm biến

Hình 2.2 thể hiện mối liên kết logic giữa các khối theo firmware. Các đường SDA/SCL nối cảm biến với ESP32; LCD dùng nhóm chân dữ liệu và điều khiển riêng; còi nhận tín hiệu từ GPIO 17. USB được sử dụng trong các lần nạp và kiểm tra thiết bị trên máy tính.



Sơ đồ liên kết tín hiệu theo firmware; các khối dùng chung GND.
Điện áp nguồn, mức logic và phân đầu nối nguồn cần đối chiếu module thực tế.

Hình 2.2. Liên kết tín hiệu phần cứng

Nguồn: sơ đồ/biểu đồ tổng hợp từ dự án [1].

Cách gắn cảm biến có ảnh hưởng trực tiếp đến tín hiệu. Khi cảm biến chỉ cố định trên breadboard, một cú gõ vào breadboard có thể tạo đáp ứng khác với cú va chạm lên vỏ hộp có lớp đệm. Vì vậy, mô hình phần cứng hiện tại thích hợp để kiểm tra giao tiếp, luồng đo và chức năng phần mềm; điều kiện này phải được giữ trong mô tả dữ liệu.

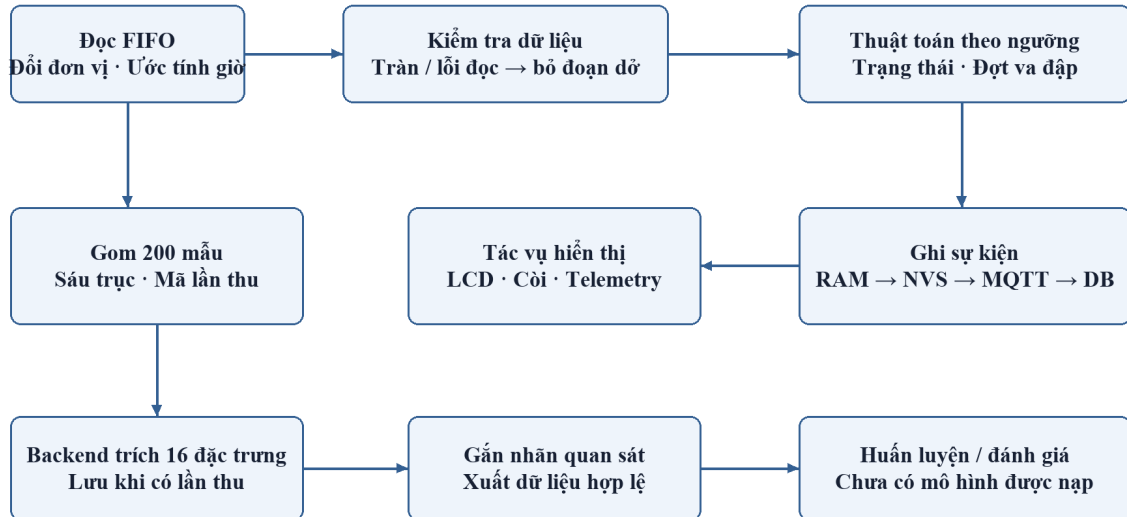
Thiết bị lấy tư thế chuẩn khi bắt đầu theo dõi. Trong thời gian hiệu chuẩn, cần giữ bộ mạch ổn định để vector tham chiếu không bị thay đổi bởi thao tác. Sau khi hiệu chuẩn, góc nghiêng được hiệu tương đối với tư thế ban đầu, không phải góc đo từ một chuẩn cơ khí bên ngoài đã được kiểm định.

Khi phát triển thành mô hình kiện hàng, việc bố trí cảm biến cần được xác định trước thực nghiệm: vị trí gắn, hướng trục, mức cố định cơ khí và cách liên kết với vỏ kiện. Những thông tin này nên trở thành metadata của phiên thử. Nếu thay cách gắn, các ngưỡng hoặc mô hình học từ cách gắn cũ cần được đánh giá lại.

Ở giai đoạn hiện tại chưa có ảnh hoặc bản đo kích thước của hộp kiện thực tế để đưa vào báo cáo. Hình 2.2 là sơ đồ liên kết, không phải ảnh minh chứng đã hoàn thành một thiết bị thương mại hoặc mạch PCB.

2.3. Thiết kế firmware và luồng lấy mẫu

Firmware sử dụng ESP-IDF và phân công các tác vụ để việc đọc cảm biến ít bị ảnh hưởng bởi LCD, còi hoặc truyền mạng. Các gói FIFO được đọc theo lô, đổi sang đơn vị đo, gán thời gian ước tính rồi đưa vào bộ phát hiện. Hình 2.3 thể hiện hai nhánh xử lý: sự kiện theo ngưỡng và các đoạn dữ liệu phục vụ AI.



Hình 2.3. Luồng xử lý mẫu và sự kiện

Nguồn: sơ đồ/biểu đồ tổng hợp từ dự án [1].

Mỗi gói FIFO có 14 byte gồm gia tốc, nhiệt độ và tốc độ góc. Phần nhiệt độ trong gói không được sử dụng làm chỉ số giám sát của dashboard. Với cấu hình đang dùng, thời gian mẫu được ước tính từ thứ tự gói, nhịp danh định và đồng hồ ESP32; điểm neo được lùi để tránh tạo thời gian nằm trong tương lai khi có lệch pha.

Nếu gặp tràn FIFO, lỗi I²C hoặc bất đồng nhịp vượt giới hạn, firmware bỏ phần đoạn AI chưa hoàn chỉnh. Hệ thống không nội suy các trục để làm một đoạn lỗi trở thành hợp lệ. Đoạn hoàn chỉnh chỉ được backend chấp nhận khi cấu trúc và các điều kiện thời gian phù hợp.

Các thông tin dành cho LCD và telemetry được chuyển qua hàng đợi; dữ liệu trực tiếp ưu tiên đoạn mới nhất khi truyền không theo kịp. Nhánh thu có mã lần thu riêng và receipt theo số đoạn. Việc dừng thu hủy phần chưa gửi/xác nhận của lượt trên thiết bị, trong khi các đoạn máy chủ đã nhận vẫn tồn tại để xem và gắn nhãn.

2.3.1. Phát hiện trạng thái và gom đợt va đập

Bộ phát hiện sử dụng tư thế chuẩn, độ lớn gia tốc, góc tương đối và RMS rung để xác định trạng thái. Một số trạng thái cần duy trì đủ số mẫu nhằm hạn chế phản ứng với một mẫu đơn lẻ. Các tham số chính được liệt kê trong Bảng 2.3; đây là tham số thử nghiệm trong mã nguồn, chưa phải ngưỡng hư hại của sản phẩm.

Bảng 2.3. Ngưỡng và điều kiện trong bộ phát hiện

Trạng thái / điều kiện	Giá trị đang dùng
Va đập – Nhẹ	Đỉnh gia tốc từ 2,5 g đến dưới 4 g.
Va đập – Trung bình	Từ 4 g đến dưới 7 g.
Va đập – Mạnh	Từ 7 g trở lên.
Rơi tự do	Gia tốc tổng hợp dưới 0,35 g; xác nhận 5 mẫu.
Nghiêng	Góc từ 35°; xác nhận 50 mẫu.
Lật	Góc từ 140°; xác nhận 50 mẫu.
Rung lắc	RMS vượt 0,12 g trong cửa sổ 50 mẫu.
Kết thúc đợt va đập	Dưới 1,8 g liên tục 80 ms; giới hạn đợt 1 giây.

Nguồn: tổng hợp từ mã nguồn và dữ liệu dự án [1].

Khi một đợt va đập bắt đầu, firmware giữ lại đỉnh gia tốc và mức cao nhất đã quan sát. Khi đợt kết thúc, kết quả được dùng thống nhất cho bản ghi, LCD và còi. Cách gom đợt tránh tạo một chuỗi sự kiện chỉ vì nhiều mẫu liên tiếp cùng nằm trên ngưỡng.

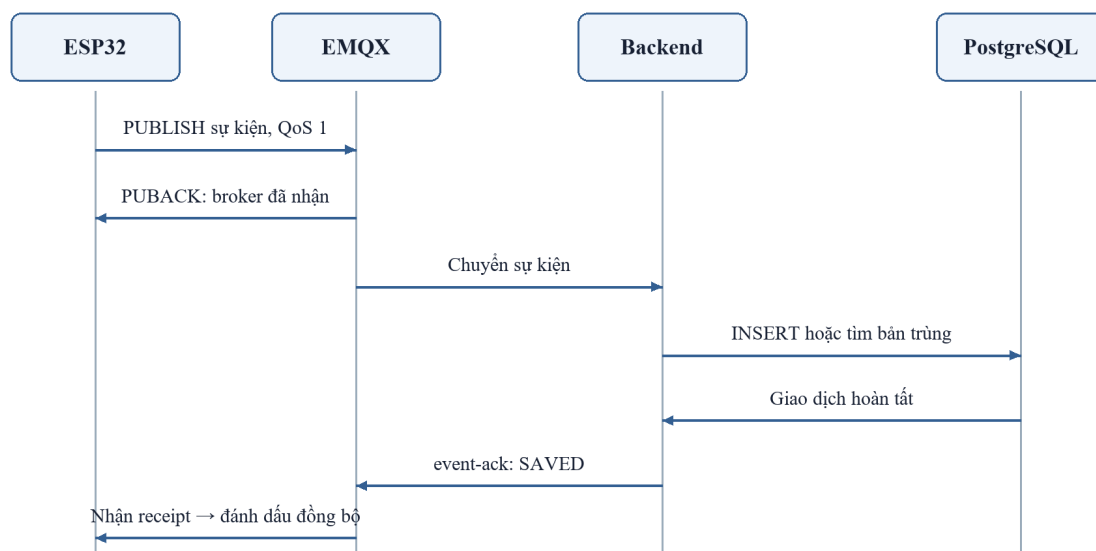
Trạng thái rơi kèm va đập được nhận biết khi trước cú va đập đã có dấu hiệu rơi tự do trong khoảng theo dõi của thuật toán. Việc nhận biết phụ thuộc vào các mẫu thu được; một chuyển động có xung quá ngắn hoặc không có pha rơi tự do rõ có thể không đáp ứng điều kiện này.

Thời lượng lưu là khoảng từ mẫu kích hoạt đầu đến mẫu cuối vượt ngưỡng duy trì, cộng một chu kỳ danh định. Nó không tương đương phép đo chính xác độ rộng của xung va đập analog. Cờ chạm giới hạn gia tốc xét từng trục gần 15,9 g, giúp nhận diện các bản ghi cần thận trọng khi diễn giải đỉnh.

Các trạng thái khác của firmware không được đồng nhất với ba lớp AI. Chẳng hạn, thông tin nghiêng hoặc lật vẫn có ý nghĩa trên dashboard nhưng mô hình AI hiện được thiết kế riêng cho ba nhóm chuyển động đã xác định.

2.4. Truyền sự kiện và xác nhận lưu dữ liệu

Hình 2.4 mô tả trình tự một sự kiện được gửi từ ESP32 đến database. Bản ghi có mã thiết bị và ID tăng dần, được đưa vào NVS trước khi tham gia cơ chế gửi lại. Backend kiểm tra nội dung, lưu sự kiện hoặc xác định bản đã tồn tại, sau đó mới phát receipt SAVED khi giao dịch hoàn tất.



Hình 2.4. Trình tự xác nhận lưu sự kiện

Nguồn: sơ đồ/biểu đồ tổng hợp từ dự án [1].

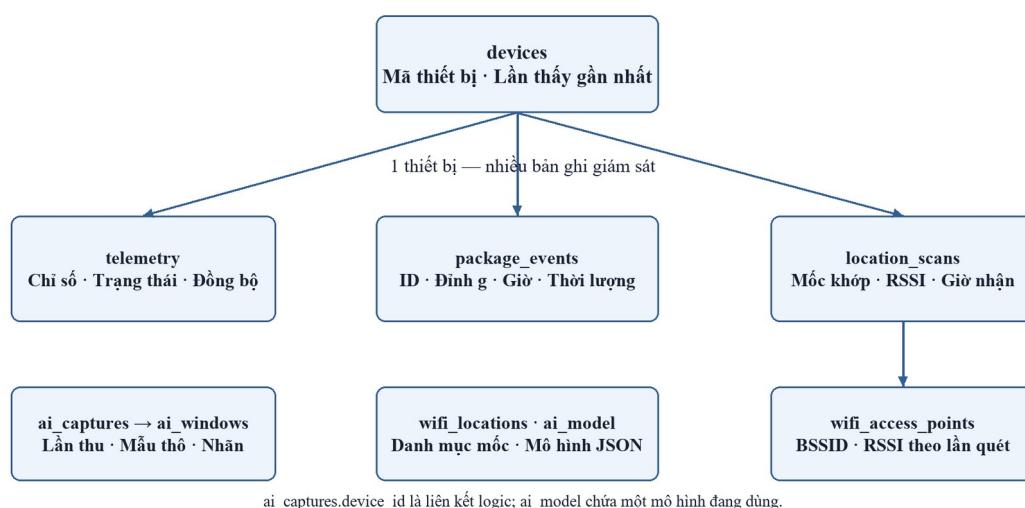
Bản tin sự kiện gồm loại trạng thái, mức thuật toán, đỉnh gia tốc, góc, RMS rung, thời gian thiết bị, uptime, thời lượng và cờ chậm giới hạn. Database tách giờ xảy ra trên thiết bị với giờ nhận tại máy chủ. Khi thiết bị chưa đồng bộ NTP, giờ xảy ra được thể hiện là chưa có, không được thay bằng giờ nhận để giả lập thời điểm thật.

Receipt đi trên topic event-ack, chứa deviceId, eventId và trạng thái SAVED. ESP32 gửi lại sau khoảng 10 giây nếu chưa nhận xác nhận phù hợp. Bản trùng không làm tăng số hàng sự kiện, nhờ khóa nghiệp vụ trên tổ hợp mã thiết bị và ID. Nâng cấp ứng dụng phải giữ ID và NVS để không tái sử dụng các khóa đã có trên máy chủ.

NVS giữ tối đa 128 sự kiện, hàng đợi ghi RAM có 32 bản ghi. Khi đầy, bản đã được xác nhận có thể được thay thế, còn bản chưa đồng bộ được bảo toàn. Tổng các lần từ chối được đưa vào telemetry; mất nguồn trước khi hàng đợi RAM được ghi xong vẫn có thể làm mất phần đang chờ. Thiết kế có giới hạn dung lượng, không bảo đảm ghi vô hạn khi mất mạng kéo dài.

2.5. Thiết kế dữ liệu và API

Dữ liệu được tổ chức theo mục đích sử dụng thay vì lưu mọi bản tin trong một bảng chung. Hình 2.5 tóm tắt các nhóm thực thể. `devices` quản lý mã thiết bị và lần nhận gần nhất; `telemetry` lưu chỉ số tức thời; `package_events` lưu các sự kiện để tra cứu; `location_scans` và `wifi_access_points` lưu kết quả quét và các điểm truy cập quan sát được.



Hình 2.5. Các nhóm dữ liệu của hệ thống

Nguồn: sơ đồ/biểu đồ tổng hợp từ dự án [1].

`wifi_locations` là danh mục móc với BSSID, tên và tọa độ. Backend chọn móc đã đăng ký phù hợp với lần quét, dựa trên điểm truy cập được nhận diện và RSSI. Nếu không khớp, hệ thống báo chưa xác định thay vì trả tọa độ cũ như một vị trí mới. Tọa độ móc là dữ liệu người dùng cấu hình, không phải tọa độ GPS do ESP32 đo.

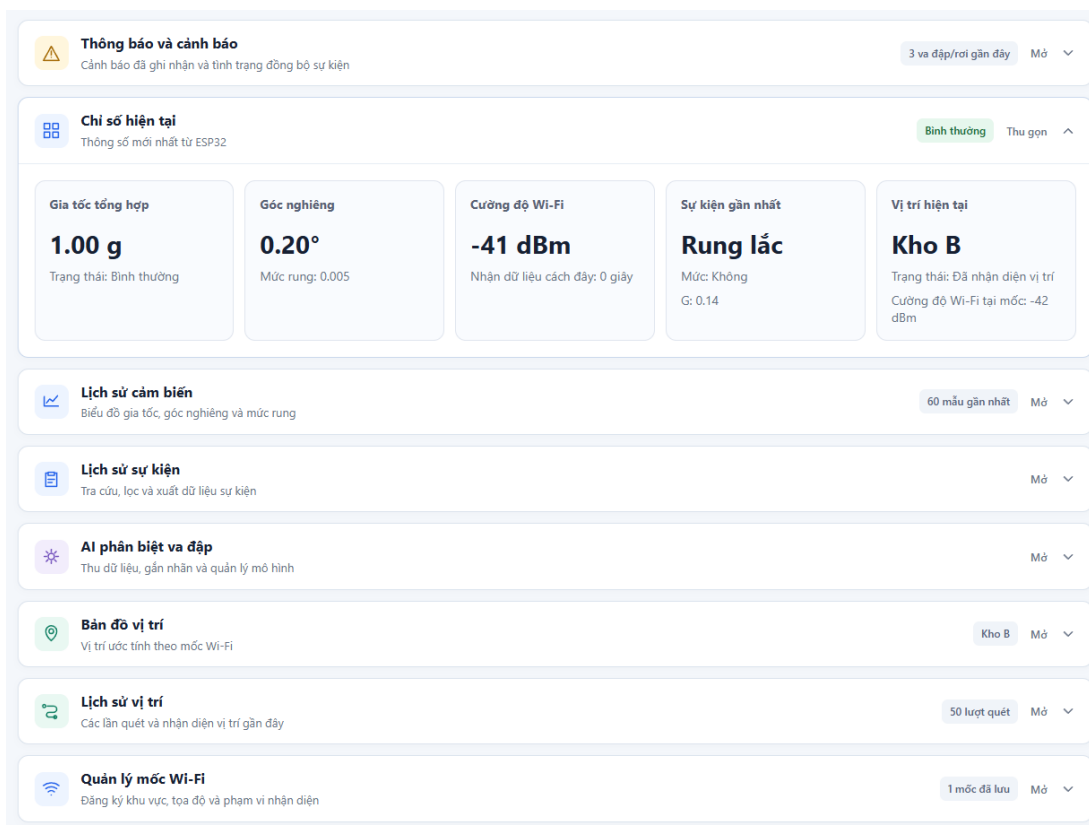
`ai_captures` lưu tên, nhóm buổi thử, thời điểm tạo và số đoạn dự kiến. `ai_windows` lưu 200 mẫu thô, 16 đặc trưng, nhãn quan sát, nhãn quy tắc và trạng thái chất lượng. Liên kết `ai_windows` với `ai_captures` được ánh xạ bằng khóa ngoại; `device_id` trong `ai_captures` là liên kết logic theo mã thiết bị trong triển khai hiện tại. `ai_model` lưu tài liệu JSON của mô hình đang dùng.

REST API phục vụ dashboard, danh sách thiết bị, lịch sử cảm biến, sự kiện, thống kê, CSV và các chức năng AI. API tra cứu có bộ lọc và phân trang; việc xuất CSV giới hạn 10.000 sự kiện để tránh một yêu cầu quá lớn. API kiểm tra sức khỏe HTTP, database và MQTT được phân biệt vì tiến trình HTTP chạy chưa chứng minh hai dịch vụ còn lại hoạt động.

Các thao tác tạo phiên, gắn nhãn, xóa dữ liệu hoặc nạp mô hình cần phiên đăng nhập và CSRF. Trong việc xem dữ liệu AI, backend kiểm tra đoạn hoặc lần thu thuộc đúng mã thiết bị được yêu cầu. Dữ liệu không hợp lệ trả lỗi có cấu trúc để frontend hiển thị và người vận hành xử lý.

2.6. Thiết kế giao diện giám sát

Giao diện được trình bày bằng tiếng Việt, có đăng nhập và tự cập nhật mỗi 3 giây. Hình 2.6 là ảnh chụp chức năng đang chạy, cho thấy các vùng được tổ chức thành thanh tiêu đề mở/đóng. Các chỉ số chính được mở mặc định; thông tin tóm tắt trên thanh giúp tìm chức năng khi nội dung đã thu gọn.



Hình 2.6. Dashboard với các mục thu gọn

Nguồn: ảnh giao diện của hệ thống [1].

Tám mục gồm thông báo, chỉ số hiện tại, lịch sử cảm biến, lịch sử sự kiện, AI, bản đồ, lịch sử vị trí và mốc Wi-Fi. Trạng thái mở/đóng được nhớ trong trình duyệt. Sau lần mở đầu, nội dung được giữ khi thu gọn để biểu mẫu AI, bộ lọc và phần đang nhập không bị khởi tạo lại.

Biểu đồ dùng dữ liệu cảm biến gần nhất; lịch sử sự kiện có lọc loại, mức, khoảng thời gian và xuất CSV. Thông báo nêu rõ sự kiện đã ghi nhận khác với trạng thái tức thời. Xác nhận đã xem chỉ thu gọn thông báo; lịch sử database vẫn được giữ và các thay đổi mới vẫn có thể tạo cảnh báo.

Giao diện hỗ trợ bàn phím qua nút tiêu đề có trạng thái mở rộng, và bố cục được kiểm tra ở màn hình điện thoại. Bản đồ điều chỉnh lại kích thước khi mở; bảng rộng được cuộn trong khu vực nội dung thay vì làm tràn toàn trang.

2.7. Quy trình dữ liệu và mô hình AI dự kiến

Một đoạn đo AI có đúng 200 mẫu, mỗi mẫu gồm thời gian và sáu giá trị gia tốc/tốc độ góc. Backend tạo 16 đặc trưng theo cùng phiên bản để công cụ Python và mô hình Java dùng chung thứ tự đầu vào. Bảng 2.4 nhóm các đặc trưng theo ý nghĩa; tên trường kỹ thuật vẫn giữ ổn định trong dữ liệu trao đổi.

Bảng 2.4. Nhóm đặc trưng đầu vào AI

Nhóm	Các đặc trưng	Ý nghĩa
Biên độ gia tốc	peak_g, min_g, mean_g	Đỉnh, nhỏ nhất và trung bình; đơn vị g.
Biến thiên gia tốc	std_g, rms_dynamic_g	Độ lệch chuẩn và RMS quanh 1 g.
Chuyển động góc	gyro_rms_dps, gyro_peak_dps	RMS và đỉnh tốc độ góc; đơn vị °/s.
Biến thiên nhanh	jerk_peak_gps	Biến thiên gia tốc lớn nhất; đơn vị g/s.
Thời gian theo ngưỡng	above_impact_ms, below_freefall_ms	Tổng khoảng mẫu trên/dưới ngưỡng.
Biến thiên từng trục	ax_std, ay_std, az_std	Độ lệch chuẩn gia tốc X, Y, Z.
Chất lượng đoạn	window_ms, sample_period_ms, saturated_ratio	Thời lượng, chu kỳ trung bình, tỷ lệ chạm giới hạn.

Nguồn: tổng hợp từ mã nguồn và dữ liệu dự án [1].

Các đoạn có khoảng cách mẫu lớn hơn 30 ms, thời lượng ngoài 1,75–2,5 giây hoặc có mẫu chạm giới hạn không được chọn vào dữ liệu huấn luyện. Nhân quan sát chỉ được gán sau xác nhận thao tác; đoạn chưa rõ tiếp tục ở trạng thái chưa gán nhãn. Trong đợt thu này còn lọc theo chế độ FIFO V3 để tránh trộn với dữ liệu polling cũ.

train.py cấu hình Random Forest 32 cây, độ sâu tối đa 6, tối thiểu 2 mẫu tại lá, cân bằng trọng số lớp và seed 42. Tách nhóm sử dụng GroupShuffleSplit với test_size = 0,34. Công cụ hiện yêu cầu ít nhất 30 đoạn và 3 nhóm cho mỗi loại; đây là điều kiện bảo vệ của triển khai, không phải định luật bảo đảm mô hình sẽ chính xác.

Mô hình xuất chứa cây, thứ tự lớp, ngưỡng điểm và báo cáo tách nhóm. Backend đối chiếu các vector kiểm tra Python–Java trước khi nhận mô hình. Ngưỡng điểm mặc định 0,7 được chọn trước đánh giá; điểm này chưa được hiệu chuẩn thành xác suất. Hiện chưa có mô hình nạp nên dashboard hiển thị Chưa huấn luyện, còn cảnh báo theo ngưỡng vẫn hoạt động.

CHƯƠNG 3. TRIỂN KHAI, THỬ NGHIỆM VÀ ĐÁNH GIÁ HỆ THỐNG

3.1. Môi trường triển khai và phương pháp thử

Hệ thống được triển khai trên máy Windows, sử dụng PowerShell để thực thi lệnh và Docker Desktop để chạy các dịch vụ. Năm dịch vụ trong cấu hình Compose gồm PostgreSQL, EMQX, backend, web và cloudflared. Backend sử dụng Java 17; firmware được build bằng ESP-IDF 5.5.5. ESP32-S3 kết nối máy tính qua cổng COM3 trong các lần nạp và kiểm tra đã ghi nhận [1].

Trình tự triển khai gồm kiểm tra cấu hình môi trường, khởi động database và broker, chạy backend, phục vụ giao diện, sau đó kiểm tra thiết bị gửi dữ liệu. Dashboard tại localhost:8088 được kiểm tra với phiên đăng nhập. Tên miền công khai sử dụng Cloudflare Tunnel và Access; kiểm tra kỹ thuật ghi nhận chuyển hướng xác thực, chưa kiểm tra đầy đủ dashboard công khai bằng phiên Access của người dùng.

Các lần cập nhật firmware liên quan đến nhíp lấy mẫu được nạp riêng vùng ứng dụng và xác minh sau nạp. Cách thực hiện giữ cấu hình mạng cùng dữ liệu NVS hiện có. Bản sao lưu trước thay đổi được giữ trong thư mục build/backups của firmware. Những thao tác này phục vụ khả năng khôi phục; chúng không thay thế phép thử mất điện hoặc kiểm tra độ bền ghi dài hạn.

Mẫu thử vật lý là bộ mạch breadboard khoảng 100 g, có ESP32, LCD, cảm biến và buzzer. Người thực hiện trực tiếp giữ yên, lắc trong tay hoặc gõ nhẹ vào cạnh nhựa bằng đầu ngón tay theo hướng dẫn. Cảm biến chưa được gắn vào vỏ một kiện hàng hoàn chỉnh. Vì vậy, kết quả chỉ mô tả đáp ứng của bộ mạch ở điều kiện thử đã nêu.

Mỗi lượt chính kéo dài khoảng 30 giây, tương ứng dự kiến 15 đoạn, mỗi đoạn 200 mẫu. Trước lượt thu, người thực hiện xác nhận sẵn sàng; sau lượt thu, xác nhận thao tác thực tế. Nhấn Bình thường, Rung lắc hoặc Va đập chỉ được ghi khi có xác nhận, không tự sao chép nhấn từ bộ phát hiện theo ngưỡng. Lượt có gián đoạn hoặc không nhớ rõ được giữ chưa gắn nhãn.

Phiên thứ hai có bước đặt lại bộ mạch và để yên ít nhất 20 giây trước khi thử. Tuy nhiên, các lượt vẫn dùng một cảm biến, cùng ngày và môi trường gần nhau. Phân nhóm theo phiên hỗ trợ tổ chức dữ liệu, chưa chứng minh tính độc lập như các ngày thử, loại hộp hoặc thiết bị khác nhau. Quá trình thu dừng theo yêu cầu của người thực hiện; báo cáo không bổ sung lượt thử giả để đủ số lượng.

Đánh giá gồm ba lớp: kiểm thử logic phần mềm bằng dữ liệu kiểm thử; quan sát kết nối và giao diện trên hệ thống chạy thật; thống kê các đoạn cảm biến có thao tác được xác nhận. Các lớp này được trình bày riêng để tránh dùng kết quả kiểm thử chương trình làm kết luận về độ chính xác phân loại vật lý.

3.2. Kết quả kiểm thử phần mềm và vận hành

Bảng 3.1 tổng hợp tám lớp JUnit tại smart_package_backend/target/surefire-reports: 28 trường hợp đạt, không có lỗi hoặc thất bại [1]. Phép thử kiểm tra điều kiện trong mã, chưa bao quát mọi tình huống vận chuyển.

Bảng 3.1. Kết quả kiểm thử tự động của backend

Lớp kiểm thử	Số ca	Kết quả
ForestModelTests	4	Đạt
MotionFeaturesTests	4	Đạt
MotionServiceTests	4	Đạt
WifiLocationRequestValidationTests	3	Đạt
MqttSubscriberTests	3	Đạt
DeviceDataServiceTests	2	Đạt
MqttMessageServiceTests	4	Đạt
WifiAnchorServiceTests	4	Đạt
Tổng	28	0 lỗi, 0 thất bại

Nguồn: tổng hợp từ mã nguồn và dữ liệu dự án [1].

Công cụ Python có bốn kiểm thử về chia nhóm, xuất cây, từ chối dữ liệu giả và điều kiện chất lượng. Ba chương trình kiểm thử C kiểm tra bộ phát hiện, lưu sự kiện và đọc FIFO. Các bản build firmware, backend và frontend đã hoàn thành theo nhật ký dự án. Dữ liệu giả trong kiểm thử mô hình được đánh dấu riêng và không được dùng để nạp mô hình vận hành.

Kiểm tra tích hợp đã phân biệt PUBACK của broker với receipt SAVED sau transaction. Khi thao tác lưu database thất bại trong phép thử, backend không phát receipt thành công; sau phục hồi, dữ liệu gửi lại được lưu. Khóa duy nhất ngăn bản gửi lại tạo sự kiện trùng. Đây là bằng chứng cho đường xử lý đã thử, chưa phải bảo đảm không mất dữ liệu khi mọi bộ đệm đều đầy.

Hai vấn đề thu đã được sửa: giảm thời gian chờ MQTT từ 1.000 xuống 50 ms để hạn chế tồn đọng truyền, và chuyển đọc polling sang FIFO. Sau nạp, sáu đoạn đứng yên đạt nhịp, không bão hòa; serial 10 giây không báo lỗi FIFO. Đây là kiểm tra vận hành khi yên.

Dashboard đã được kiểm tra đăng nhập, cập nhật, biểu đồ, bộ lọc và tám vùng thu gọn trên máy tính cùng màn hình 375 px, không ghi nhận lỗi JavaScript. Bộ đếm lịch sử 277 được giữ nguyên; pendingEvents bằng 0 ở các mẫu kiểm tra. Thông báo được sửa để không hiểu số tích lũy là bộ nhớ đang đầy.

3.3. Kết quả thu dữ liệu cảm biến

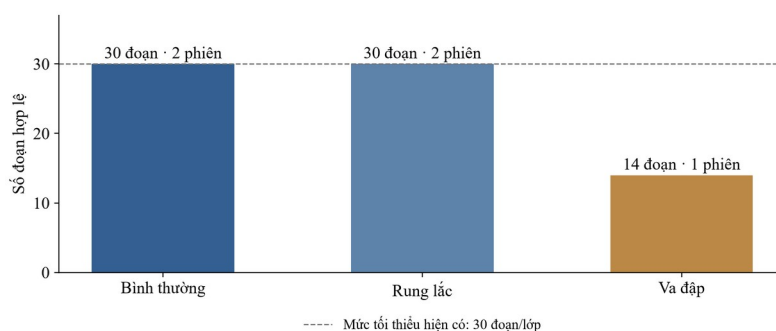
Bộ dữ liệu được chọn chỉ gồm các lượt đã xác nhận thao tác, cùng chế độ FIFO_100HZ_ESTIMATED_TIME_V3. Bảng 3.2 thể hiện năm lượt chính. Mỗi lượt nhận 15 đoạn; một đoạn Va đập bão hòa bị loại khỏi huấn luyện nhưng vẫn giữ bản đo trong database [1].

Bảng 3.2. Các lượt thu thực tế được lựa chọn

Loại / phiên	Thời gian	Đã lưu	Hợp lệ
Va đập – phiên 1	30 giây	15	14
Bình thường – phiên 1	30 giây	15	15
Rung lắc – phiên 1	30 giây	15	15
Bình thường – phiên 2	30 giây	15	15
Rung lắc – phiên 2	30 giây	15	15
Tổng	150 giây	75	74

Nguồn: tổng hợp từ mã nguồn và dữ liệu dự án [1].

Tổng cộng 75 đoạn lưu có 15.000 mẫu; 74 đoạn hợp lệ chứa 14.800 mẫu. Thời gian mỗi đoạn hợp lệ theo đặc trưng xuất nằm khoảng 1.977–1.987 ms. Vì thời gian mẫu FIFO được ước tính, dải này phản ánh cửa sổ dữ liệu do phần mềm xây dựng, không phải sai số thời gian đo bằng thiết bị chuẩn độc lập.



Hình 3.1. Số đoạn hợp lệ theo nhãn quan sát

Nguồn: sơ đồ/biểu đồ tổng hợp từ dự án [1].

Hình 3.1 cho thấy Bình thường và Rung lắc đều có 30 đoạn từ hai phiên; Va đập có 14 đoạn từ một phiên. Đường 30 đoạn thể hiện điều kiện số lượng hiện có của công cụ, không phải một chuẩn khoa học về lượng dữ liệu đủ. Số nhóm tối thiểu vẫn chưa đạt ở cả ba lớp.

Lượt thăm dò 10 giây lưu bốn đoạn, có gián đoạn nên giữ chưa gắn nhãn. Dữ liệu polling cũ được bảo toàn nhưng không nằm trong tập 74 đoạn. Màn hình AI tổng hợp toàn database có thể hiển thị số khác; cần lọc đúng chế độ và mã lần thu khi đối chiếu.

3.4. Đánh giá kết quả và giới hạn

Có thể đối chiếu nhãn quy tắc trên từng đoạn với nhãn thao tác đã xác nhận. Bảng 3.3 là thống kê mô tả trên 74 đoạn đang có; cột thể hiện nhãn do quy tắc trả về, hàng thể hiện nhãn quan sát. Đây không phải ma trận đánh giá một mô hình AI đã huấn luyện hay kết quả trên tập kiểm tra độc lập.

Bảng 3.3. Đối chiếu nhãn quan sát với nhãn quy tắc

Nhãn quan sát	Bình thường	Rung lắc	Va đập
Bình thường (30)	30	0	0
Rung lắc (30)	3	25	2
Va đập (14)	2	1	11

Nguồn: tổng hợp từ mã nguồn và dữ liệu dự án [1].

Hai nhãn trùng nhau ở 66/74 đoạn, tương ứng khoảng 89,2% trên dữ liệu đã thu. Tỷ lệ này chỉ diễn tả mức trùng nhãn trong phạm vi thử, không được ghi là độ chính xác AI. Năm đoạn rung không trùng nhãn và ba đoạn gõ không được quy tắc xếp Va đập cho thấy ngưỡng cố định còn phụ thuộc cách thao tác. Nhãn cả đoạn cũng không mô tả chính xác thời điểm từng cú gõ bên trong.

Công cụ huấn luyện yêu cầu ít nhất 30 đoạn hợp lệ và ba nhóm cho mỗi lớp. Tập hiện tại có 30/2, 30/2 và 14/1 theo thứ tự Bình thường, Rung lắc, Va đập. Điều kiện chưa đạt nên công cụ từ chối huấn luyện; hệ thống chưa có mô hình AI đang hoạt động. Không có cơ sở để báo accuracy, precision, recall, F1 hoặc thời gian suy luận của một mô hình vận hành.

Về phần cứng, tần số danh định 100 Hz có thể bỏ lỡ xung va chạm rất ngắn; bão hòa làm mất thông tin về đỉnh thật. Cảm biến gắn trên breadboard không tái hiện cấu trúc vỏ hộp, đệm và hàng hóa. Dữ liệu chỉ từ một thiết bị và một ngày thử, nên chưa đánh giá độ bền theo thời gian, khác biệt lắp đặt hay khả năng áp dụng cho kiện có khối lượng khác.

Về truyền và lưu, NVS 128 sự kiện cùng hàng đợi RAM 32 phần tử đều hữu hạn. Khi mất mạng dài hoặc tốc độ sự kiện cao, hệ thống vẫn có thể từ chối dữ liệu mới. Bộ đếm lịch sử 277 ghi nhận giới hạn đã xảy ra, không thể phục hồi các sự kiện chưa từng được lưu. Cơ chế gửi lại và receipt giúp kiểm soát đồng bộ các bản còn giữ được.

Kết quả phù hợp để trình diễn chuỗi cảm biến – truyền – lưu – giám sát và quy trình chuẩn bị AI. Hệ thống chưa được xác nhận là công cụ đánh giá hư hại hàng hóa hoặc sản phẩm vận chuyển thương mại. Bước phát triển có giá trị nhất là chuẩn hóa cách gắn cảm biến và thu thêm điều kiện thử khác nhau, sau đó mới đánh giá phân loại trên nhóm chưa dùng để huấn luyện.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Đề tài đã xây dựng được một mô hình giám sát dùng ESP32-S3 và cảm biến chuyển động, kết hợp MQTT, backend Java, PostgreSQL cùng dashboard tiếng Việt. Thiết bị xử lý trạng thái theo ngưỡng, hiển thị LCD và phát còi; phía máy chủ lưu lịch sử, hỗ trợ tra cứu, xuất dữ liệu và hiển thị vị trí theo mốc Wi-Fi khi có cấu hình phù hợp. Các vùng chức năng có thể thu gọn để thuận tiện theo dõi.

Luồng sự kiện có ID, lưu NVS, gửi lại và receipt sau khi database lưu thành công. Thông báo đã được điều chỉnh để phân biệt lịch sử không lưu được với số đang chờ đồng bộ. Kết quả kiểm thử phần mềm và vận hành chứng minh các đường xử lý được nêu trong Chương 3; vẫn cần phép thử dài hạn và các kịch bản nguồn, mạng, dung lượng trước khi mở rộng phạm vi sử dụng.

Phần AI đã có công cụ thu đoạn, kiểm tra chất lượng, gắn nhãn quan sát, trích 16 đặc trưng, huấn luyện và xuất mô hình dự kiến. Bộ dữ liệu được lựa chọn có 74 đoạn hợp lệ, từ năm lượt trên breadboard. Vì chưa đạt điều kiện số đoạn và số nhóm, mô hình chưa được huấn luyện hoặc nạp. Báo cáo không xem tỷ lệ trùng nhãn quy tắc là kết quả AI.

Hướng phát triển trước tiên là đặt bộ mạch trong hộp, cố định cảm biến lên thành hoặc đáy hộp và mô tả khối lượng, vật liệu, lớp đệm cùng phương pháp tác động. Cần thu các phiên riêng theo ngày, cách lắp và điều kiện vận chuyển, bảo đảm ba nhân được người thực hiện xác nhận. Dữ liệu không rõ thao tác hoặc chất lượng kém tiếp tục được giữ để đối chiếu nhưng không đưa vào huấn luyện.

Khi có dữ liệu phù hợp, cần chia nhóm huấn luyện và kiểm tra, đánh giá từng lớp bằng ma trận nhầm lẫn cùng precision, recall và F1. Ngưỡng chưa chắc chắn, phiên bản đặc trưng và mô hình phải được ghi trong hồ sơ đánh giá. Mô hình chỉ nên được kích hoạt sau khi kiểm tra tính nhất quán giữa Python và Java; cảnh báo tại thiết bị vẫn cần hoạt động khi máy chủ hoặc mô hình không sẵn sàng.

Các bước tiếp theo khác gồm so sánh tín hiệu với thiết bị đo tham chiếu, xem xét nhịp lấy mẫu cao hơn cho xung ngắn, kiểm thử mất mạng/mất điện và theo dõi dung lượng database. Khi mở rộng nhiều thiết bị, cần đánh giá tải, phân quyền và quản lý cấu hình theo từng mã thiết bị. Những nội dung này là hướng đề xuất, không được tính vào chức năng đã kiểm chứng của phiên bản hiện tại.

TÀI LIỆU THAM KHẢO

- [1] Nhóm 9, “Mã nguồn, nhật ký kiểm thử và dữ liệu thử nghiệm của hệ thống Giám sát va đập kiện hàng,” tài liệu nội bộ dự án, đối chiếu ngày 06/10/2026.

Tư liệu đối chiếu: `firmware-esp32/main/`; `smart_package_backend/target/surefire-reports/`; `docs/TESTING.md`; `ai-training/thu-du-lieu.local/phien.json`, `du-lieu-ai.json` và `tong-ket-thu-du-lieu.txt`.

- [2] Espressif Systems, “ESP32-S3 Series Datasheet,” phiên bản 2.2, Espressif Documentation. [Trực tuyến]. Có tại: https://documentation.espressif.com/esp32_s3_datasheet_en.pdf. [Truy cập: 06/10/2026].

- [3] TDK InvenSense, “MPU-6500,” InvenSense Products. [Trực tuyến]. Có tại: <https://www.invensense.tdk.com/en-us/products/inertial-sensors/mpu-6500>. [Truy cập: 06/10/2026].

- [4] OASIS, “MQTT Version 3.1.1,” OASIS Standard, 2014. [Trực tuyến]. Có tại: <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>. [Truy cập: 06/10/2026].

- [5] Espressif Systems, “Non-Volatile Storage Library,” ESP-IDF Programming Guide, v5.5.5. [Trực tuyến]. Có tại: https://docs.espressif.com/projects/esp-idf/en/v5.5.5/esp32s3/api-reference/storage/nvs_flash.html. [Truy cập: 06/10/2026].

TÀI LIỆU THAM KHẢO (TIẾP THEO)

- [6] Spring, “Cross Site Request Forgery (CSRF),” Spring Security Reference. [Trực tuyến]. Có tại: <https://docs.spring.io/spring-security/reference/servlet/exploits/csrf.html>. [Truy cập: 06/10/2026].
- [7] PostgreSQL Global Development Group, “5.5. Constraints,” PostgreSQL 17 Documentation. [Trực tuyến]. Có tại: <https://www.postgresql.org/docs/17/ddl-constraints.html>. [Truy cập: 06/10/2026].
- [8] Cloudflare, “Cloudflare Tunnel,” Cloudflare Docs. [Trực tuyến]. Có tại: <https://developers.cloudflare.com/tunnel/>. [Truy cập: 06/10/2026].
- [9] scikit-learn developers, “RandomForestClassifier,” scikit-learn Documentation. [Trực tuyến]. Có tại: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>. [Truy cập: 06/10/2026].
- [10] scikit-learn developers, “GroupShuffleSplit,” scikit-learn Documentation. [Trực tuyến]. Có tại: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GroupShuffleSplit.html. [Truy cập: 06/10/2026].

PHỤ LỤC. HƯỚNG DẪN DEMO VÀ ĐỐI CHIẾU

A. Trình tự trình diễn

1. Khởi động Docker Desktop và các dịch vụ theo hướng dẫn của dự án; cấp nguồn ESP32. Đăng nhập dashboard tại địa chỉ triển khai hoặc localhost:8088 bằng tài khoản đã cấu hình.
2. Chọn esp32-001, quan sát trạng thái kết nối cùng thời gian cập nhật. Mở “Chỉ số hiện tại” và “Lịch sử cảm biến”; giữ bộ mạch yên để kiểm tra dữ liệu tiếp tục thay đổi theo thời gian.
3. Mở “Lịch sử sự kiện”, dùng bộ lọc và xuất CSV nếu cần. Đối chiếu “Thông báo” với số đang chờ đồng bộ; xác nhận đã xem chỉ tác động trạng thái thông báo, không xóa bản ghi.
4. Mở khu vực AI, xem các lần thu và mẫu đã gán nhãn. Trạng thái chưa có mô hình là kết quả hiện tại. Chỉ bắt đầu thu khi người thực hiện sẵn sàng và xác nhận thao tác sau khi kết thúc.
5. Mở bản đồ và mốc Wi-Fi để giải thích cách khớp điểm truy cập. Không trình diễn điểm hiển thị như tọa độ GPS của kiện hàng.

B. Tập phục vụ kiểm tra báo cáo

Số đoạn và nhóm: ai-training/thu-du-lieu.local/tong-ket-thu-du-lieu.txt. Đặc trưng và nhãn từng đoạn: du-lieu-ai.json trong cùng thư mục. Kết quả kiểm thử backend: các tệp TEST-*.xml tại smart_package_backend/target/surefire-reports. Nhật ký vận hành: docs/TESTING.md.

Không đưa mật khẩu, token hoặc nội dung .env vào báo cáo, tệp trình chiếu hay bản demo công khai. Ảnh phần cứng trong báo cáo là sơ đồ liên kết tín hiệu; ảnh giao diện là ảnh chụp hệ thống, không phải ảnh dựng sản phẩm.

C. Phạm vi công cụ hỗ trợ

Công cụ AI được sử dụng để hỗ trợ rà soát mã nguồn, tổng hợp nội dung, dựng sơ đồ và dàn trang báo cáo. Các số liệu được đối chiếu với tệp dự án; thao tác vật lý do người thực hiện xác nhận. Nhóm cần kiểm tra, giải thích được nội dung và bổ sung thông tin [] trước khi nộp, theo yêu cầu của giảng viên.